

accoding OJ交互题制作流程

基于 SPJ 与原生运行监控器的实时交互适配

版本：1.0 验证日期：2026-09-14 适用范围：本次实测的 accoding 评测镜像、C/C++ 提交

01 实现结果与运行原理

本流程已在实验题 10296「寻找最大值（实时交互实验）」上验证。最终 C 与 C++ 标程均通过全部 36 组数据；错误答案、询问超限、未刷新输出、答对后异常退出四种提交在单点实验中均判为 WA。

OJ 没有原生交互器配置，但 SPJ 执行时仍能通过原生运行监控器启动当前提交。因此，让普通评测先完成一次极短的预运行，再由 SPJ 启动正式交互。正式阶段的每次回复都根据当前询问计算，选手可以根据回复决定下一次询问。

阶段	执行内容	结果
1. 普通预运行	OJ 编译提交，向程序输入 0；程序正常退出	普通评测继续调用 SPJ
2. 读取私有数据	SPJ 从答案文件读取真实 n 和隐藏排列	隐藏排列保留在 SPJ 内
3. 启动正式交互	SPJ 调用 <code>/utils/process_monitor</code> ，再次启动 <code>/test/main</code>	以选手身份运行，接入两条 FIFO
4. 实时问答	SPJ 发送真实 n，接收 ?l r 并立即回复	选手自适应发出下一次询问
5. 最终判定	检查 !p、询问次数、运行结果和退出码	向 OJ 返回得分与固定结果标签

两个启动实例互不共享程序内存。预运行只负责让 OJ 进入 SPJ 阶段；所有求解逻辑都在第二次启动后完成。SPJ 不依赖第一次运行的输出记录完成正式交互。

原生监控器负责将选手降为 uid/gid 8194 并执行资源限制。适配器没有直接以 SPJ 的 root 身份运行提交，也没有修改后台脚本、运行器或权限配置。

使用边界

这是依赖当前镜像内部接口的实验适配，并非平台承诺的稳定交互 API。当前只验证了 C/C++。更换镜像后必须重新验证可执行文件位置、监控器参数与运行身份。页面耗时和内存来自预运行，正式交互的资源限制由内层监控器执行。

实验题入口：[10296](#)。本题未加入课程比赛。

02 准备工程并确认平台接口

配套工程位于工作区的 `实时交互实验_寻找最大值/`。以下命令均在该工程目录执行；本文中的工程文件名也相对于此目录。

文件或目录	用途
<code>compare_interactive.cpp</code>	已验证的完整 C++14 SPJ，粘贴到后台代码框
<code>problem.md</code> 、 <code>solution.md</code> 、 <code>answer.c</code>	题面、题解与可提交的 C 标程
<code>data/</code>	OJ 专用数据：36 对 <code>.in/.ans</code> ，另有源码与 <code>makefile</code>
<code>local_data/</code> 、 <code>run_local.py</code>	常规本地双向管道交互环境
<code>generate_data.py</code> 、 <code>verify.py</code>	生成数据，核验算法、回复和协议
<code>test_native_adapter.py</code>	一次性 Linux 容器中的适配器通信层测试
<code>negative_submissions/</code>	错误答案、超次、未刷新、异常退出提交
<code>spj_interactive.zip</code>	OJ 数据包，共 74 个文件
<code>接入实验记录.md</code> 、 <code>oj_experiments.json</code>	实验结论与平台证据索引

旧说明与当前后台的区别

用户提供的旧说明要求在 `/data` 执行 `make`，生成 `compare`。本次后台的实际接入路径不同，不能仅上传 `makefile` 就认为交互器已启动。

项目	本次实测情况
数据上传位置	<code>/data/data/</code>
SPJ 构建入口	后台高级设置的 SPJ 代码框，实际按 C++14 编译
SPJ 可执行文件	<code>/test/__special_compare__</code>
当前提交可执行文件	<code>/test/main</code> ，调用 SPJ 时仍然存在
可复用的运行监控器	<code>/utils/process_monitor</code>
默认选手标准输入	普通文件；发出询问后读取不会自动得到回复

`data/makefile` 与 `data/compare.cpp` 保留了旧说明要求的源码包形式，执行 `make` 可生成 `compare`。本次成功接入采用 SPJ 代码框。上传的 `makefile` 没有作为自动预启动钩子执行。

在另一个镜像复用时，应先确认上述接口，再做单点交互验证。完整 SPJ 在运行器或选手可执行文件不存在时直接判失败，不会改为绕过监控器执行提交。

03 设计题目、协议与数据

示例题隐藏一个 $1..n$ 的排列， $2 \leq n \leq 100000$ 。选手每次询问一个区间，得到该区间第二大元素的下标，需要在最多 40 次询问内找出全局最大值的下标。题目改编自 CF 1486C1，原题约 1600，接近本次约 1500 的目标。

明确写入题面的协议

方向	内容	规则
OJ → 选手，预运行	0	立即正常退出，退出码为 0
SPJ → 选手，正式开始	n	后续只通过交互获取信息
选手 → SPJ，询问	? l r	$1 \leq l < r \leq n$ ，不超过 40 次
SPJ → 选手，回复	一个整数	区间第二大元素在原排列中的下标
选手 → SPJ，答案	! p	最大值下标；随后正常退出
SPJ → 选手，错误	-1	读到后立即退出；EOF 时同样退出

每条命令以换行结尾；每次询问和最终答案输出后都要刷新标准输出。调试信息不要写入标准输出。最终答案后只允许空白字符，不能继续输出询问或其他文字。

OJ 文件格式

每个公开输入文件，例如 casea.in，只有预运行标记：

```
0
```

对应私有答案文件 casea.ans 的第一行是 n，第二行是隐藏排列。例如：

```
5
2 5 1 4 3
```

该例正式启动后 SPJ 先发送 5；收到 ? 1 5 时回复 4；最终正确答案为 ! 2。上述数组仅用于说明文件格式，不应出现在普通输入文件中。

生成并检查数据

运行 python3 generate_data.py。脚本同时生成 data/ 和 local_data/，并同步 SPJ 打包副本。本地数据的公开输入是真实 n，其 .ans 只保存排列；上传 OJ 必须使用 data/。36 组的 OJ 名称为 casea 至 caseaj，对应关系记录在 data_manifest.json。

04 编写支持预运行的选手程序

以下为工程中的完整 C 标程，同时可按 C++ 编译。它先处理 $n == 0$ ，每次输出后刷新，读到 EOF 或 -1 时退出。

```
#include <stdio.h>
#include <stdlib.h>

int ask(int l, int r) {
    if (l == r) return 0;
    printf("? %d %d\n", l, r);
    fflush(stdout);
    int s;
    if (scanf("%d", &s) != 1) exit(0);
    if (s == -1) exit(0);
    return s;
}

int main(void) {
    int n;
    if (scanf("%d", &n) != 1 || n == 0) return 0;
    int l = 1, r = n;
    while (l < r) {
        int s = ask(l, r);
        int mid = (l + r) / 2;
        if (s <= mid) {
            if (ask(l, mid) == s) {
                r = mid;
            } else {
                l = mid + 1;
            }
        } else {
            if (ask(mid + 1, r) == s) {
                l = mid + 1;
            } else {
                r = mid;
            }
        }
    }
    printf("! %d\n", l);
    fflush(stdout);
    return 0;
}
```

每轮先询问候选区间，再询问包含第二大元素的半区。如果回复仍是原下标，最大值也在该半区；否则在另一半区。单点半区无需实际询问。候选长度每轮至少约减半，最多 $2 \times \text{ceil}(\log_2 n) \leq 34$ 次询问，小于限制 40。

05 实现 SPJ 的实时交互控制

完整可用代码为 `compare_interactive.cpp`。以下说明其关键环节，复用时应复制完整源码，不能只拼接启动片段。

读取私有数据并建立通信

SPJ 保留标准的三个文件路径参数：`argv[1]` 是普通输入，`argv[2]` 是私有答案，`argv[3]` 是预运行输出。它校验输入标记为 0，校验隐藏排列的范围、完整性及额外内容，然后关闭数据文件。第三个文件不作为正式交互的输出来源。

在 `/test` 下用 `mkdtemp` 创建独立目录，目录权限为 `0700`；创建两条权限为 `0600` 的 FIFO。父进程使用非阻塞读写，并将真实 `n` 写入选手输入通道。临时目录不会向选手开放。

通过原生监控器启动提交

子进程建立独立会话，将监控器报告重定向至私有文件，然后调用以下代码。输入、输出和错误路径均由当前 SPJ 创建。

```
execl("/utils/process_monitor", "process_monitor",
      "-t", "1000", "-m", "65536", "-p", "5",
      "-uid", "8194", "-gid", "8194",
      "-inf", in_path.c_str(),
      "-outf", out_path.c_str(),
      "-errf", error_path.c_str(),
      "/test/main", (char *)NULL);
```

监控器在降低选手身份前打开重定向文件；选手只使用标准输入输出。参数使用 `execl` 传递，没有通过 shell 拼接执行命令。

实时解析、判定与收尾

1. 逐行解析？`!r` 或 `!p`，拒绝未知操作、额外字段、越界、空字节和过长命令。当前单行最多 127 个非换行字符。
2. 每次有效询问扫描隐藏区间，计算第二大元素下标，立即回写。第 41 次询问失败。
3. 验证最终答案，拒绝答案后的非空白内容。总墙钟等待为 3 秒；错误时尝试发送 `-1`。
4. 等待监控器收尾；错误后的额外清理等待最多 1500 ms，必要时终止对应进程组，并回收子进程。
5. 只有答案正确、协议正常、监控器正常退出且报告中的 `Run result code` 与 `Exit code` 均为 0 才接受。删除 FIFO、日志和临时目录。

SPJ 成功输出两行 `1、Accepted`；失败输出两行 `0、Wrong Answer`。即使答案正确，提交随后异常退出也应失败。

06 在 accoding 后台配置

按顺序完成配置

1. 创建实验题，填入题面、交互说明和预运行规则。已有验证题可在题目 10296 的编辑页面核对配置。
2. 在高级设置启用 SPJ，将完整 `compare_interactive.cpp` 粘贴到代码框，选择 C++。本次后台实际以 C++14 编译。
3. 从 `data/` 成对添加同名 `.in` 与 `.ans`，确认每个输入文件仅含 0。首次接入先添加一个小测试点。
4. 配置外层时间和内存，保存后提交标程。单点成功、反例被拒绝后，再添加全部 36 组。
5. 完整数据上线后，分别用 C 与 C++ 提交标程，确认每个测试点都通过。

参数位置	本次值	实际约束对象
后台单组时间	500 ms	普通预运行
后台内存	65536 KB	普通预运行
36 组外层总时间	18000 ms	需满足后台总时间上限 30000 ms
SPJ 监控器 <code>-t</code>	1000 ms	正式交互的选手程序
SPJ 监控器 <code>-m</code>	65536 KB	正式交互的选手程序
SPJ 监控器 <code>-p</code>	5	正式交互的选手进程数
SPJ 墙钟等待	3 秒	防止交互一直等待

外层设置与内层限制需要分别维护。在后台把时间改为 2000 ms，不会自动改变 SPJ 中的 `-t 1000`。更换题目时应同时核对题面和两层配置。

三个已经遇到的配置问题

总时间超限。本次 36 组最初填写单组 1000 ms，合计 36000 ms，被后台拒绝。改为外层单组 500 ms 后，合计 18000 ms，正式交互仍是内层每组 1000 ms。

替换文件未更新引用。已有测试点若改用不同文件名，应删除旧测试点后新增。本次仅上传新名字的替换文件，没有更新原有测试点引用。

附加说明导致 OE。当前后台对失败说明的处理与旧描述不完全一致。生产版本保持第二行为 `Wrong Answer`；启用 `SPJ_DIAGNOSTIC` 后的附加诊断可能使页面显示 Other Error，得分仍为 0。诊断结束后恢复默认输出。

07 从本地核验到真实 OJ 验收

本地验证

在工程目录执行以下命令，先生成数据，再检查交互轨迹与完整测试结果：

```
python3 generate_data.py
python3 run_local.py --case 03 --log demo_transcript.txt
python3 verify.py
```

测试自己的已编译程序：

```
python3 run_local.py --case 03 -- /absolute/path/to/program
```

现有记录通过了 36 组数据、872 个小排列、4416 次回复的独立排序核验、19 项协议异常和 4 组地址/未定义行为检查；标程实测最多询问 33 次。结果见 [verification.json](#)。

适配器通信层另有 [test_native_adapter.py](#)。它会创建 `/test` 与 `/utils/process_monitor` 替身，因此只在一次性 Linux 容器运行。以下镜像是本地现有镜像；其他机器需替换为包含 Python 3 和 GCC/G++ 的 Linux 镜像。

```
docker run --rm --platform linux/arm64 --network none \
-v "$PWD:/work:ro" ai-dl-tauri-ubuntu-arm64:22.04 \
python3 /work/test_native_adapter.py
```

该测试通过 36 组正例、11 项异常、合法零询问答案、预运行退出和 FIFO 清理检查。它使用运行器替身，仅能验证通信层，真实运行器兼容性必须以 OJ 提交为准。

真实 OJ 证据

提交编号	验证内容	实测结果
8185636	首次完整单点实时交互	AC
8185647	最终 C 标程，全部 36 组	36/36 AC，得分 1
8185648	最终 C++ 标程，全部 36 组	36/36 AC，得分 1
8185637	错误最终答案，单点	WA，得分 0
8185638	第 41 次询问，单点	WA，得分 0
8185639	未刷新询问便等待回复，单点	WA，得分 0
8185640	答对后以退出码 7 结束，单点	WA，得分 0

原始记录分别保存在 `live_full_results.json` 和 `live_negative_results.json`。本次没有声称四种反例都在完整 36 点版本上测试。

08 排错与制作下一道交互题

常见现象及处理

现象	优先检查
程序第一次询问后读到 EOF	是否处理 <code>n == 0</code> ；是否已启用本适配 SPJ
本地通过、OJ 不进入交互	是否误传 <code>local_data/</code> ；SPJ 是否粘贴完整；运行器接口是否匹配
询问后一直等待，最后 WA	是否换行并刷新；命令格式和范围是否有效
答案正确但 WA	是否存在多余输出、未换行、非零退出或监控器拒绝
保存完整数据后出现 OE	外层时间总和是否超过 30000 ms；是否仍开诊断输出
更换数据后仍像在测旧样例	测试点引用的实际文件名是否更新
页面显示耗时极小	属于预运行统计，不能据此判断正式算法性能
上传 makefile 没有启动服务	本次后台通过 SPJ 代码框构建，按第 06 节配置

复用修改什么

保留预运行入口、原生监控器启动、FIFO 通信、超时清理、退出状态检查和固定判定标签。根据新题修改私有数据格式与校验、公开初始信息、询问语法、回复计算、询问上限和最终答案判定。同步修改数据生成器、标程、题面、题解与异常测试。

若新题的询问计算量明显增加，还需重新评估 SPJ 自身开销与墙钟等待。当前回复采用逐段扫描，适用于本题的规模与询问次数；不能未经评估就照搬到更大数据或更多询问。

发布前验收清单

- 题面说明输入 0 的预运行、输出刷新、EOF/-1 退出、正式限制及页面统计含义。
- 所有 OJ 输入只含预运行标记，私有数据格式正确；后台 SPJ 与工程源码一致。
- 单点标程 AC，错误答案、超限、缺少刷新和异常退出被拒绝。
- 全量数据下允许的每种语言均通过；记录提交编号和测试日期。
- 关闭诊断附加输出，检查数据引用、外层总时间与临时资源清理。
- 更换镜像或适配接口后重复上述验证，再用于正式题目。

配套文件与依据

本文整理自本次工程源码、[README.md](#)、[接入实验记录.md](#) 及本地和 OJ 的验证记录。可从本文所在目录访问配套工程说明、完整 SPJ 源码和数据包。移动本文时，请保留工程目录或同步调整相对链接。

算法来源：[Codeforces 1486C1](#)。本文记录的是 2026-09-14 实测版本；Accoding 的课程难度仍待评，实验题尚未加入课程比赛。